

QML

September 17, 2026

[]: Week-02 (Day-09): QUANTUM MACHINE LEARNING -RAJIV KUMAR YADAV

```
[10]: import qiskit
import numpy as np

from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator

print("Qiskit:", qiskit.__version__)
print("NumPy:", np.__version__)

# =====
# 1. DATASET
# =====

X = np.array([
    [0.1, 0.2],
    [0.2, 0.1],
    [2.8, 2.9],
    [2.9, 2.8]
])

y = np.array([
    0,
    0,
    1,
    1
])

# =====
# 2. SCALE FEATURES
# =====

X_scaled = X / 3.0 * np.pi
```

```

# =====
# 3. QUANTUM CIRCUIT
# =====

def create_circuit(features, weights):

    qc = QuantumCircuit(2, 1)

    # -----
    # Feature encoding
    # -----

    qc.ry(features[0], 0)
    qc.ry(features[1], 1)

    # -----
    # Trainable layer
    # -----

    qc.ry(weights[0], 0)
    qc.ry(weights[1], 1)

    # Entanglement
    qc.cx(0, 1)

    # -----
    # Second trainable layer
    # -----

    qc.ry(weights[2], 0)
    qc.ry(weights[3], 1)

    # Measure qubit 0
    qc.measure(0, 0)

    return qc

# =====
# 4. QUANTUM PROBABILITY
# =====

simulator = AerSimulator()

def probability_one(features, weights):

```

```

qc = create_circuit(
    features,
    weights
)

compiled = transpile(
    qc,
    simulator
)

result = simulator.run(
    compiled,
    shots=1000
).result()

counts = result.get_counts()

ones = counts.get("1", 0)

probability = ones / 1000

return probability

# =====
# 5. COST FUNCTION
#
# Instead of:
#
# prediction = 0 or 1
#
# we use:
#
# probability = 0.0 ... 1.0
# =====

def cost_function(weights):

    total_error = 0

    for features, label in zip(
        X_scaled,
        y
    ):

        probability = probability_one(

```

```

        features,
        weights
    )

    error = (
        probability - label
    ) ** 2

    total_error += error

    return total_error / len(y)

# =====
# 6. INITIAL PARAMETERS
# =====

np.random.seed(42)

weights = np.random.uniform(
    -np.pi,
    np.pi,
    4
)

print("\nInitial parameters:")
print(weights)

initial_cost = cost_function(
    weights
)

print(
    "Initial cost:",
    round(initial_cost, 4)
)

# =====
# 7. TRAINING
# =====

best_weights = weights.copy()

best_cost = initial_cost

```

```

for iteration in range(200):

    # Create small random variation
    candidate = (
        best_weights
        + np.random.normal(
            0,
            0.2,
            4
        )
    )

    candidate_cost = cost_function(
        candidate
    )

    # Keep candidate if better
    if candidate_cost < best_cost:

        best_weights = candidate.copy()

        best_cost = candidate_cost

    if iteration % 20 == 0:

        print(
            "Iteration:",
            iteration,
            "Cost:",
            round(best_cost, 4)
        )

# =====
# 8. PREDICTIONS
# =====

predictions = []
probabilities = []

for features in X_scaled:

    probability = probability_one(
        features,
        best_weights
    )

```

```

        probabilities.append(
            probability
        )

        if probability >= 0.5:
            prediction = 1
        else:
            prediction = 0

        predictions.append(
            prediction
        )

# =====
# 9. ACCURACY
# =====

predictions = np.array(
    predictions
)

accuracy = np.mean(
    predictions == y
)

# =====
# 10. RESULTS
# =====

print("\n=====")
print("          QML CLASSIFICATION")
print("=====")

print(
    "Actual labels:      ",
    y
)

print(
    "Probabilities:      ",
    np.round(probabilities, 3)
)

print(

```

```

        "Predicted labels: ",
        predictions
    )

    print(
        "Accuracy: ",
        round(
            accuracy * 100,
            2
        ),
        "%"
    )

    print(
        "Final cost: ",
        round(
            best_cost,
            4
        )
    )

# =====
# 11. DISPLAY TRAINED CIRCUIT
# =====

print("\nTrained quantum circuit:\n")

example = create_circuit(
    X_scaled[0],
    best_weights
)

print(
    example.draw(
        output="text"
    )
)

```

Qiskit: 2.4.1

NumPy: 2.4.6

Initial parameters:

`[-0.78828768 2.83192151 1.45766093 0.61988954]`

Initial cost: 0.2884

Iteration: 0 Cost: 0.2884

Iteration: 20 Cost: 0.0053

Iteration: 40 Cost: 0.0001

```

Iteration: 60 Cost: 0.0
Iteration: 80 Cost: 0.0
Iteration: 100 Cost: 0.0
Iteration: 120 Cost: 0.0
Iteration: 140 Cost: 0.0
Iteration: 160 Cost: 0.0
Iteration: 180 Cost: 0.0

```

```
=====
```

QML CLASSIFICATION

```
=====
```

```

Actual labels:      [0 0 1 1]
Probabilities:      [0.008 0.007 0.993 0.999]
Predicted labels:   [0 0 1 1]
Accuracy:           100.0 %
Final cost:         0.0

```

Trained quantum circuit:

```
q_0: Ry( /30) Ry(-0.0038599) Ry(-0.055431) M
```

```
q_1: Ry( /15) Ry(2.7476) X Ry(0.45303)
```

```
c: 1/
```

0

[]: